

**Statistics 6910:
Getting Started with R**

Why  ?

- Some of you may have used statistical software with a GUI, like Minitab or SPSS. You may also be familiar with other programming languages, like C, Java, Python, etc.
- We will use the R programming language and environment as our “home base” for performing many data analytic tasks.

Why  ?

- Allows custom analysis
- High-level scripting language
- Statistical programming language
- Interactive exploratory data analysis
- Easy to replicate analysis
- Sound numerical methods
- Large Community of contributors

Why  ?

It's Free!

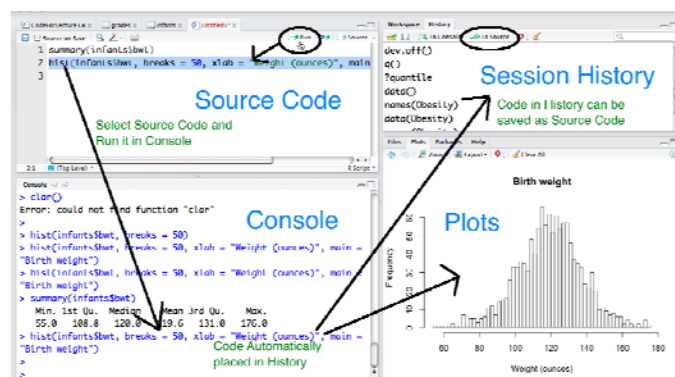
Let's Install R

- Open a Web browser and go to Google:
<http://www.google.com/>
- Search for R (that's right – the letter R).
- One of the top links will be to [The R Project for Statistical Computing](http://www.r-project.org/), which takes you to [http:// www.r-project.org/](http://www.r-project.org/)
- At <http://www.r-project.org> click on CRAN (left menu)
- Select a mirror site near us, i.e., there is a mirror site at <http://cran.stat.ucla.edu>

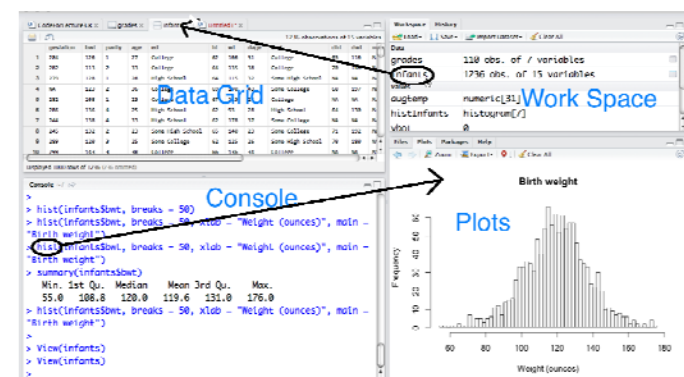
Let's Install R Studio

- Open a Web browser and go to Google:
<http://www.google.com/>
- Search for RStudio
- One of the top links will be <http://www.rstudio.com/> (click on Download tab)
- Download the version for running R on your desktop

Let's Try It - RStudio



Let's Try It - RStudio



The Prompt

- The R prompt is: `>`
- At the prompt, type an **expression**

```
> 2 + 3
Returns 5
```
- Hit the return/enter key

```
> rnorm(3)
Returns 3 random
normal values
0.1603903 -0.2925857 -
0.8274805
```
- R **evaluates** the expression (performs a **computation**)
- R returns a value

```
> hist(x)
Returns a plot
```

What do expressions look like?

```
2 + 3
9 - 8
4 * 5
10 / 3
7 ^ 2
9 %/% 2
11 %% 7
```

Parsing Expressions

- How does R know what computation to perform?
- It breaks down an expression into parts, called **tokens**; these are elementary building blocks that can't be broken down any further
- From these pieces it can figure out what computation to perform

Parsing English

hatheads

- The above letters are the beginning of something that I am writing.
- Can you figure out what it is?
- What would make it easier for you to do this?

How do we Parse English?

- Punctuation: ., ! ? : ;
- Capitalization
- Blank spaces

So What Does hatheads... mean?

Ha! The ad's finished!

"Hat!" He ads on his way out the door.

How do we Parse in R?

- White space: **22** vs **2 2**
- Atomic Tokens **+ - * / ^**
; end of line
- Quotation Marks **#** comment
"Hi" or **'Bye'** not **"My"**
- Naming Conventions **x2** not **2x**
- New Line

How do we Parse in R?

- New Line


```
(2 + 34)
[1] 36
(2 + 3
4)
Error: unexpected
numeric constant
in:
"(2 + 3
4"
(2 + 34
)
[1] 36
```

Order of Operations

Order of operations is what you expect:

- exponentiation first, followed by multiplication and division, then addition and subtraction;
- left to right;
- parentheses override order

To Do

- Write the following as an R expression:

`10^(15 / 3 - 2)`

`power(10, subtract(divide(15,3),2))`

- Write the following as an R expression

$$\frac{\sqrt{6-2}}{3^2}$$

`(6-2)^0.5 / 3^2`

or

`sqrt(6-2) / 3^2`

- Circle the tokens in the following R expression

`cat = (1 + x2)^24`

`cat = (1 + x2)^24`
 ^ ^ ^ ^ ^ ^ ^ ^ ^

We all make mistakes in writing code

Understanding how R parses expressions will help you fix them

Output and Assignment

When we evaluate an expression, R prints the results to the screen as output

- How do we save the result?
- How do we use the output as input to another expression?

Output and Assignment

- We can assign the result of the computation to a variable, e.g., named x:

`> x = 10^(15 / 3 - 2)`

- We can use `x` as an input in another expression

`> sqrt(x)`

- To see the value of a variable type the variable name at the prompt and hit return

`> x`

Output and Assignment

- `=` and `<-` are both valid assignment operators
- `x <- 10^(15 / 3 - 2)` is the same as
`x = 10^(15 / 3 - 2)`
- Choose one and use it consistently!
- As we'll see, `==` means something completely different.

Variables

- Variables have a name and a value
- To access the value we use the name
- Variables allow us to:
 - Store a value without needing to recompute it
 - Write a general expression, e.g. `sqrt(a^2 + b^2)`
 - Reduce redundancy (and mistakes)

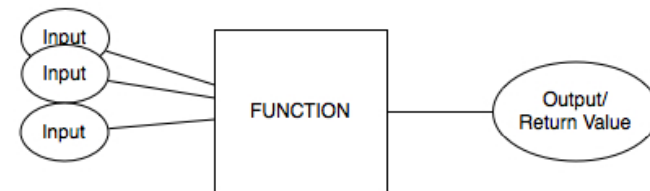
Variable Names

- Variable names must follow some rules
 - May not start with a digit or underscore (`_`)
 - May contain numbers, characters, and some punctuation - period and underscore are ok, but most others are not
 - Case-sensitive, so `x` and `X` are different
- Advice on variable names:
 - Use meaningful names
 - Avoid names that have meaning in R, e.g., function names. If in doubt, check:

```
> exists("pi")
[1] TRUE
```

Function Style Expressions

- Functions contain code (expressions) that perform a specific task.



The Inputs are called **arguments**
 The output is the **return value**

Function Style Expressions

- When you use a function with a particular set of arguments, you are said to be **calling** the function
- R **evaluates** the function call and returns the output
- For now, we will work with R's built-in functions

Examples Built-in functions

- | | |
|---------------|-----------------------|
| • log() | • mean() |
| • exp() | • sd() |
| • sqrt() | • median() |
| • abs() | • min() max() range() |
| • sin() cos() | • sum() |

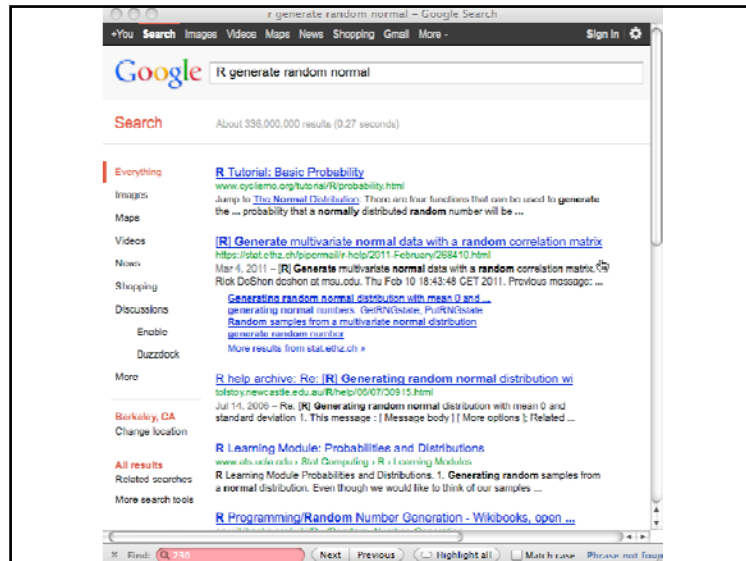
How to call a function

Suppose we want to generate 3 random values from a normal curve with center 0 and spread 5.

- Is there a function in R that can do this?
- How do we find it?
- How do we call it?

How to find it?

- Use Google search
 - I find searches like the following helpful:
R generate random normal
- R has built-in search
– `help.search("topic")`



How to call a function

- We call a function as follows:
FunctionName(argument, ..., argument)
- Functions can have one or more inputs
- Some arguments are required.
- Other arguments are optional. They have default values so you don't have to specify them.

How to call rnorm

- We can find out the arguments to rnorm:
`> args(rnorm)`
function (n, mean = 0, sd = 1)
- We see it has 3 arguments: `n`, `mean` and `sd`
- `mean` and `sd` are optional. – they have default values (0 and 1, respectively)
- `n` must be specified – it has no default
- We can learn more with the help function:
`> ?rnorm`
`> help("rnorm")`

Generate 3 random values from a normal with center 0 and spread 5

- Generate 3 normals with mean 0 and sd 1.
`rnorm(3)`
- Arguments can be identified by position:
`rnorm(3, 0, 5)`
- Arguments can be identified by name:
`rnorm(n = 3, sd = 5)`
- Use position and name:
`rnorm(3, sd = 5)` (OK for first argument but otherwise be careful)

Simple and Compound Expressions

- Simple Expression: `rnorm(3, sd = 5)`
- Compound Expression: `mean(rnorm(3))`
- Ill-formed Expressions: `mean(rnorm(3)) ]`

Can you spot what's wrong?

Error: unexpected ']' in "mean(rnorm(3))]"

Let's try out what we have
learned